

Checking if a tree is balanced ▾

I forget how do you calculate id a tree is not balanced

A tree is **not balanced** if the height difference between the left and right subtrees of any node is more than 1.

Here's the standard recursive approach:

python

```
def is_balanced(root):
    def check(node):
        if not node:
            return 0 # height of empty tree is 0

        left_height = check(node.left)
        if left_height == -1:
            return -1 # already unbalanced

        right_height = check(node.right)
        if right_height == -1:
            return -1 # already unbalanced

        if abs(left_height - right_height) > 1:
            return -1 # this node is unbalanced

        return 1 + max(left_height, right_height) # return height

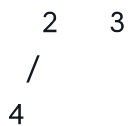
    return check(root) != -1
```

The key idea:

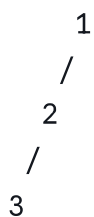
- Recursively compute the height of left and right subtrees at every node
- If the difference exceeds 1 at any node → unbalanced
- Use `-1` as a sentinel value to propagate "already found unbalanced" up the call stack

Example:

1
/ \



At node 2: left height = 1, right height = 0 $\rightarrow$ diff = 1 

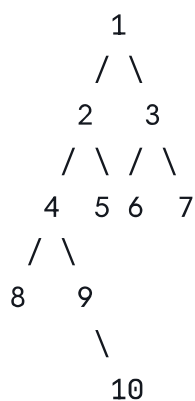


At node 1: left height = 2, right height = 0 $\rightarrow$ diff = 2  **Not balanced**

Time complexity: $O(n)$ — each node is visited once.

Can you give a more complex example?

Sure! Here's a more complex example with a larger tree:



Let's trace through the algorithm bottom-up:

Step 1 — Leaf nodes (10, 8, 5, 6, 7) all return height 1.

Step 2 — Node 9:

- left = 0, right = 1 (node 10)
- diff = 1  $\rightarrow$ returns height 2

Step 3 — Node 4:

- left = 1 (node 8), right = 2 (node 9)

- $\text{diff} = 1$ ✓ → returns height 3

Step 4 — Node 2:

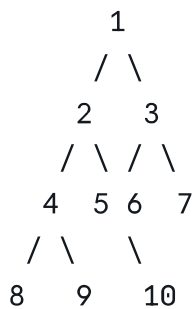
- left = 3 (node 4), right = 1 (node 5)
- $\text{diff} = 2$ ✗ → returns -1 (UNBALANCED!)

Step 5 — Node 1:

- left immediately returns -1 (propagated from node 2)
- Short-circuits, returns -1

Result: Tree is NOT balanced — the imbalance is at node 2, because its left subtree (height 3) is too much deeper than its right subtree (height 1).

To **fix it**, you'd need to either trim the left side or add nodes to the right:



Now at node 3: left = 1, right = 2 → $\text{diff} = 1$ ✓

At node 2: left = 2, right = 1 → $\text{diff} = 1$ ✓

At node 1: left = 3, right = 3 → $\text{diff} = 0$ ✓

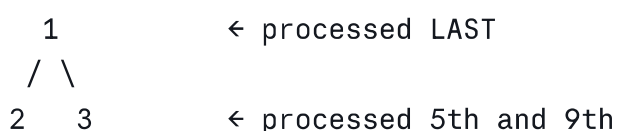
Result: Balanced! ✓

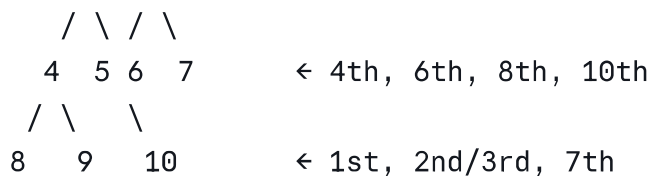
It works depth first huh?

Yes exactly! The recursion naturally goes **depth-first** (specifically post-order DFS) — it fully explores each subtree before returning results upward.

The order is: **left → right → current node**

So for the balanced tree example, the actual call order is:



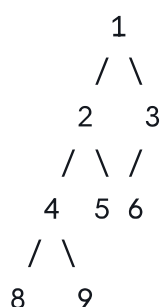


The key reason it works this way is that **a node can't know its own height until its children report theirs first**. So the leaves go first, and results bubble up.

This is also why the **-1** sentinel is efficient — once an imbalance is detected deep in the tree, every node above it just immediately passes **-1** up without doing any more work, effectively **short-circuiting** the rest of the traversal.

Use the same exmaple u gove me but remove 10 and 7

Sure! Here's the tree with 10 and 7 removed:



Tracing through:

Step 1 — Leaf nodes (8, 9, 5, 6) all return height **1**.

Step 2 — Node 4:

- left = 1 (node 8), right = 1 (node 9)
- diff = 0 ☒ → returns height **2**

Step 3 — Node 2:

- left = 2 (node 4), right = 1 (node 5)
- diff = 1 ☒ → returns height **3**

Step 4 — Node 3:

- left = 1 (node 6), right = 0 (missing)
- diff = 1 ☒ → returns height **2**

Step 5 — Node 1:

- left = 3 (node 2), right = 2 (node 3)
- diff = 1  → returns height 4

Result: Tree IS balanced! 

Every node has a left/right height difference of at most 1, so the whole tree passes.

So intuitively a tree is not balanced or skewed when the difference between each level of the leaves are more than 1?

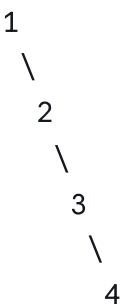
Close, but slightly more precise — it's not just about the leaves, it's about **every internal node** in the tree.

At each node, you ask: *"is the height of my left subtree vs right subtree within 1?"* If **any** single node fails that check, the whole tree is unbalanced.

A good way to think about it intuitively:

- **Balanced** → the tree looks roughly "filled out" and symmetric, no branch is dramatically longer than its sibling
- **Unbalanced/Skewed** → one side keeps going deeper while the other stops, like a lopsided tree

The worst case is a **fully skewed tree**, where every node only has one child:



This is essentially a linked list — at node 1, left height = 0, right height = 3, diff = 3 . Maximally unbalanced.

The reason this matters practically is **performance** — a balanced tree gives you $O(\log n)$ search, while a skewed tree degrades to $O(n)$, same as a linked list. That's why structures like AVL trees and Red-Black trees automatically rebalance themselves to keep that guarantee.

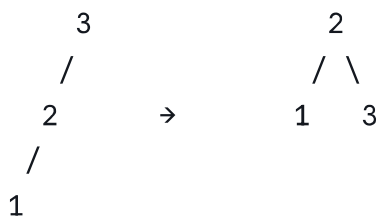
AVL how

AVL trees rebalance by performing **rotations** whenever an insertion or deletion causes a node's balance factor to exceed 1.

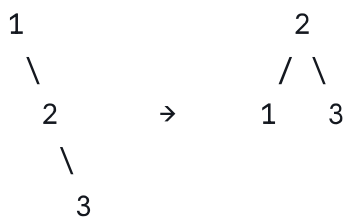
The **balance factor** of a node = $\text{height}(\text{left}) - \text{height}(\text{right})$. It must always be -1, 0, or 1.

There are 4 rotation cases:

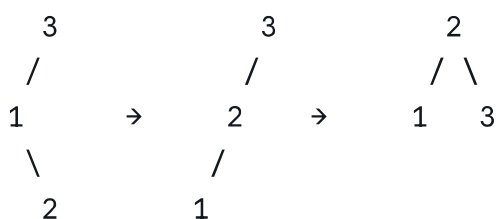
Case 1 — Left Left (rotate right):



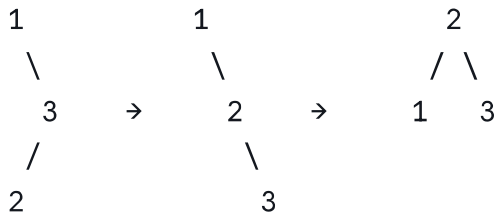
Case 2 — Right Right (rotate left):



Case 3 — Left Right (rotate left, then right):



Case 4 — Right Left (rotate right, then left):



The key insight is that **rotations preserve the BST property** ($\text{left} < \text{root} < \text{right}$) while restoring balance. After every insert/delete, the tree walks back up to the root checking balance factors, and rotates wherever needed.

Cases 1 & 2 need **one rotation**, cases 3 & 4 need **two** because the imbalance is "zig-zag" shaped rather than a straight line.

Okay but can you explain LL RR LR and RL? How to do it why does it work how to do it in a an actual tree

Sure! Let's build intuition first.

The core idea: When a tree becomes unbalanced, we need to "lift" a node up to restore balance without breaking the BST property. The rotation type depends on *where* the imbalance is.

How to identify which case you're in

After inserting, find the first unbalanced node (balance factor > 1). Then ask:

- Did the insertion go **left or right** of the unbalanced node?
- Did it then go **left or right** again?

```

LL = went Left, then Left again  → rotate Right
RR = went Right, then Right again → rotate Left
LR = went Left, then Right       → rotate Left then Right
RL = went Right, then Left       → rotate Right then Left
  
```

LL Case — Single Right Rotation

Insert 3, 2, 1 into a BST:

```

      3          ← unbalanced (balance factor = 2)
     /
    2          ← imbalance is on the Left, Left side
   /
  1

```

Fix: Lift 2 up, make 3 its right child:

```

      2
     / \
    1   3

```

Why it works: 2 was already greater than 1 and less than 3, so the BST property is preserved. We just restructured who is the root.

RR Case — Single Left Rotation

Insert 1, 2, 3:

```

  1          ← unbalanced (balance factor = -2)
   \
    2          ← imbalance is on the Right, Right side
     \
      3

```

Fix: Lift 2 up, make 1 its left child:

```

      2
     / \
    1   3

```

Same idea, just mirrored.

LR Case — Double Rotation (Left then Right)

Insert 3, 1, 2:

```

      3          ← unbalanced
     /
    1          ← went Left

```

```

  \
   2      ← then Right (zig-zag!)

```

The zig-zag shape is why one rotation isn't enough. If you just rotated right:

```

  1      ← still unbalanced!
  \
   3
  /
 2

```

Still broken. So instead you do **two steps**:

Step 1 — Rotate **1** left first to straighten it into LL shape:

```

    3
   /
  2
 /
1

```

Step 2 — Now rotate **3** right (standard LL):

```

  2
 / \
1  3

```

Why it works: The double rotation converts the zig-zag into a straight line first, then fixes it.

RL Case — Double Rotation (Right then Left)

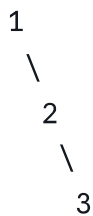
Insert 1, 3, 2:

```

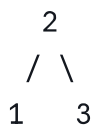
  1      ← unbalanced
  \
   3      ← went Right
  /
 2      ← then Left (zig-zag!)

```

Step 1 — Rotate **3** right first to straighten into RR shape:



Step 2 — Now rotate 1 left (standard RR):



The pattern in one sentence

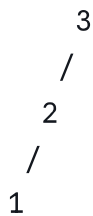
Case	Shape	Fix
LL	Straight left	Rotate right once
RR	Straight right	Rotate left once
LR	Zig-zag left-right	Rotate left to straighten, then rotate right
RL	Zig-zag right-left	Rotate right to straighten, then rotate left

Straight line = one rotation. Zig-zag = two rotations to straighten it first.

Ok sk 3left1rifht2 for LR became 1 rifht 3 left 2 cause i thought that you raise one and make 3 its child so 1left 2 and 1 right 3

You're right! Let me re-examine that.

After Step 1 (rotate 1 left):



You're correct that 2 gets lifted up, 1 becomes its left child, and 3 stays above. So it's:

- 2's left = 1
- 3's left = 2

Which gives:

```

      3
     /
    2
   /
  1

```

Then Step 2 rotates 3 right, lifting 2 up:

- 2's left = 1
- 2's right = 3

```

    2
   / \
  1   3

```

So your instinct was right — **each rotation always lifts the middle node up and makes the other a child**. I think where the confusion was is that after step 1 you get $3 \rightarrow 2 \rightarrow 1$ (a straight left line, i.e. LL), not 1 as root with 2 and 3 as children. That final shape only happens after the second rotation.

The two steps are:

1. Straighten the zig-zag into a straight line
2. Then fix the straight line with a single rotation

Wait what can you explain the LR thing again?

Sure! Let's go super slow.

We start with:

```

    3
   /

```

```

  1
   \
    2

```

This is **LR** because from **3** we went **Left** (to **1**), then **Right** (to **2**). Zig-zag shape.

Step 1 — Rotate **1** left:

"Lifting **2** up, pushing **1** down to the left"

```

    3
   /
  2
 /
1

```

So now **2** is where **1** was, and **1** becomes **2**'s left child.

Step 2 — Rotate **3** right:

"Lifting **2** up, pushing **3** down to the right"

```

    2
   / \
  1   3

```

So now **2** is the root, **1** is its left child, **3** is its right child.

The key intuition is:

- **Step 1** just straightens the zig-zag into a straight line (now it looks like LL)
- **Step 2** is then just a normal LL rotation

Does that clear it up?

Can you give me more example?

Sure! Let me give a more complex tree where LR happens.

Let's insert 7, 5, 3, 4, 6 one by one:

Insert 7, 5, 3 — triggers LL rotation:

```
  7
 /
5
/
3
```

LL rotation → lift 5:

```
  5
 / \
3   7
```

Insert 4 — no imbalance yet:

```
  5
 / \
3   7
 \
  4
```

Insert 6 — no imbalance yet:

```
  5
 / \
3   7
 \ /
 4 6
```

Okay that wasn't very exciting. Let me give one that actually **triggers LR**:

Insert 5, 3, 4:

```

  5
 /
3
 \
 4

```

From **5** we go **Left** to **3**, then **Right** to **4** — that's **LR**!

Step 1 — Rotate 3 left (lift **4** up, push **3** down):

```

  5
 /
4
 /
3

```

Step 2 — Rotate 5 right (lift **4** up, push **5** down):

```

  4
 / \
3   5

```

✅ **Balanced!**

Now a bigger one. Insert **10, 5, 8**:

```

 10
 /
 5
 \
  8

```

From **10** go **Left** to **5**, then **Right** to **8** — **LR**!

Step 1 — Rotate 5 left:

```

 10
 /
 8
 /
 5

```

Step 2 — Rotate 10 right:

```

      8
     / \
    5   10

```

✓ Balanced!

The pattern is always the same — whenever you see this **zig-zag shape** going left then right:

```

      X
     /
    A
     \
     B

```

B always ends up as the root, A goes left, X goes right:

```

      B
     / \
    A   X

```

That's the universal LR result regardless of the values!

Question, so for the code ineed to make 4 seperate fhings for each cases? Or just 2

Just **2 functions** — `rotate_left` and `rotate_right`! The 4 cases just call them in different combinations:

python

```

def rotate_right(y):
    x = y.left
    T2 = x.right

    x.right = y
    y.left = T2

```

```

    # update heights
    y.height = 1 + max(height(y.left), height(y.right))
    x.height = 1 + max(height(x.left), height(x.right))

    return x # new root

def rotate_left(x):
    y = x.right
    T2 = y.left

    y.left = x
    x.right = T2

    # update heights
    x.height = 1 + max(height(x.left), height(x.right))
    y.height = 1 + max(height(y.left), height(y.right))

    return y # new root

```

Then the 4 cases just mix and match:

```

python

# LL
if balance > 1 and key < node.left.val:
    return rotate_right(node)

# RR
if balance < -1 and key > node.right.val:
    return rotate_left(node)

# LR
if balance > 1 and key > node.left.val:
    node.left = rotate_left(node.left) # step 1
    return rotate_right(node)         # step 2

# RL
if balance < -1 and key < node.right.val:
    node.right = rotate_right(node.right) # step 1
    return rotate_left(node)              # step 2

```

So really it's:

- **LL** = `rotate_right` once
- **RR** = `rotate_left` once

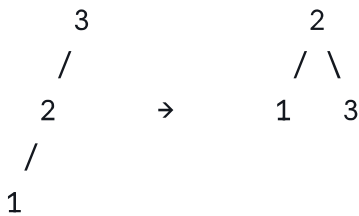
- **LR** = `rotate_left` then `rotate_right`
- **RL** = `rotate_right` then `rotate_left`

Just 2 building blocks, composed differently for each case!

Question what does rotate left and right mean?

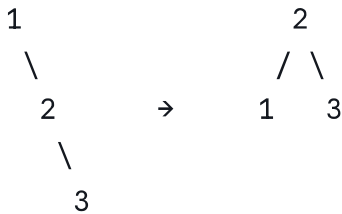
Great question! It's about which direction the **top node falls**.

Rotate Right — the top node falls to the **right**:



3 falls to the right, **2** takes its place.

Rotate Left — the top node falls to the **left**:



1 falls to the left, **2** takes its place.

The code reflects this literally:

```
python
def rotate_right(y):
    x = y.left      # x is the one moving UP

    y.right = ...   # y is falling to the RIGHT
    x.right = y     # y becomes x's right child

def rotate_left(x):
```

```

y = x.right      # y is the one moving UP

x.left = ...     # x is falling to the LEFT
y.left = x       # x becomes y's left child

```

So the name just describes **what happens to the root** — it either falls left or falls right, and its child rises up to take its place.

A simple way to remember:

- **Rotate right** = top node goes right = left child rises up
- **Rotate left** = top node goes left = right child rises up

Can you demonstrate with LR or RL

Sure! Let's use LR with **5, 3, 4**:

```

  5
 /
3
 \
 4

```

Step 1 — Rotate 3 left (3 falls left, 4 rises up):

```

  5
 /
3 ← 3 falls left
 \
 4 ← 4 rises up

```

After:

```

  5
 /
4   ← 4 took 3's spot
 /
3   ← 3 fell to become 4's left child

```

Step 2 — Rotate 5 right (5 falls right, 4 rises up):

```

      5 ← 5 falls right
    /
   4 ← 4 rises up
  /
 3

```

After:

```

    4      ← 4 took 5's spot
   / \
  3   5    ← 3 stays left, 5 fell to become 4's right child

```

The key thing to notice is that **4 appears twice** across the two steps:

- Step 1: 4 rises to take 3's spot
- Step 2: 4 rises again to take 5's spot

That's why the middle value always ends up as the root after LR — it gets lifted **twice**!

Can you make an example out of 1

```

  /\
 2 3
 /\
4 5 6 7
 /\
8 9
 \
 10

```

Sure! Let's insert a node that triggers LR. Let's insert **11** under node **9** to the... wait, that would be RR.

Let me instead insert **8.5** (between 8 and 9), so it goes under **9** as a left child:

```

    1
   /\

```



Now look at node **4**:

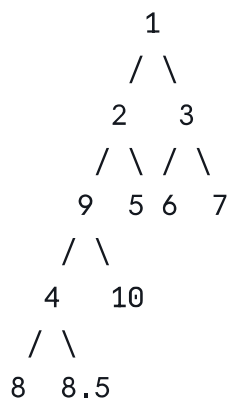
- Left height = 1 (node 8)
- Right height = 2 (node 9)
- Balance factor = -1  still fine

But now look at node **2**:

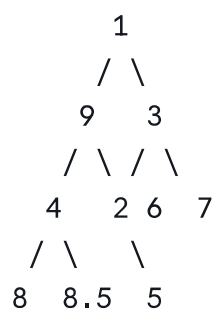
- Left height = 3 (node 4)
- Right height = 1 (node 5)
- Balance factor = 2  unbalanced!

And the path from **2** goes **Left** to **4**, then **Right** to **9** — that's **LR**!

Step 1 — Rotate **4 left** (4 falls left, 9 rises up):



Step 2 — Rotate **2 right** (2 falls right, 9 rises up):



```

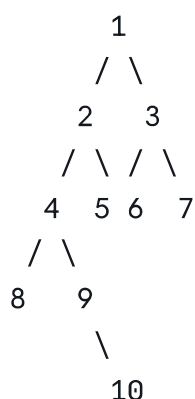
  \
10

```

✅ Balanced! Node 9 rose up twice, just like in the simple example!

Wait but if you didn't insert anything it would be unbalanced already

You're absolutely right! As we established earlier, the original tree:



Is **already unbalanced** at node 2:

- Left height = 3 (through 4 → 9 → 10)
- Right height = 1 (node 5)
- Balance factor = 2 ❌

So an AVL tree would have **never allowed this shape to exist** in the first place — it would have rebalanced as each node was inserted.

So to properly demonstrate LR on a real AVL tree, we should build it from scratch by inserting nodes one by one and letting it rebalance as we go. Want me to do that? We can insert the same values (1-10) and watch the AVL rotations happen step by step as each node gets inserted?

Oh that's why I'm so confused on how AVL do it on that right then

Yeah exactly! That's the key thing about AVL trees —

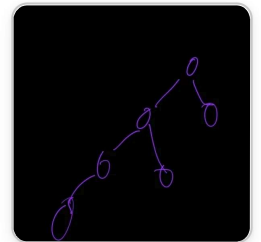
You can't just draw an arbitrary tree and apply AVL rotations to it. AVL rebalancing happens *during* insertion, one node at a time. So the tree never gets a chance to look as unbalanced as that example.

The process is always:

1. Insert a node (like normal BST)
2. Walk back up to the root checking balance factors
3. If any node is unbalanced → rotate immediately
4. Continue up

So by the time you've inserted all 10 nodes, the shape of the tree will look **completely different** from what you'd get with a plain BST insertion.

Want me to walk through inserting 1-10 one by one and show how the tree shape changes with each rotation? It'll make it much clearer how AVL actually builds the tree progressively rather than fixing it at the end.



Okay what if i have like this

Nice drawing! Let me read it — it looks like:



Is that right? I'm having a bit of trouble reading the values clearly. Can you confirm the numbers?

But regardless of the exact values, I can already see the **structure** — the left side keeps going deeper while the right side is shallow. So:

- At the root: left height = 3, right height = 1 $\rightarrow$ balance factor = 2 ✖

This would be an **LL** or **LR** case depending on where exactly the imbalance zig-zags.
What are the actual numbers?

No numbers its placeholders

Oh got it! So the structure is:



So walking up from the bottom, the first unbalanced node is the **second level O** (the left child of root):

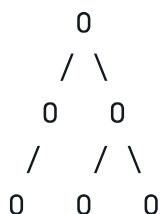
- Left height = 2 (through the bottom left O)
- Right height = 1 (the right O)
- Balance factor = 1 ✔ still fine

Then at the **root**:

- Left height = 3
- Right height = 1
- Balance factor = 2 ✖ unbalanced!

Now which case is it? From root we go **Left**, then **Left** again (the deeper side) — that's **LL**!

Rotate root right (root falls right, left child rises up):



✅ Balanced! The left child became the new root, the original root fell to the right, and everything redistributes evenly.

Does that match what you were thinking?

No can you explain more detail

Sure! Let me label them with letters so it's easier to follow:



Step 1 — Check balance factors bottom up:

- **F:** no children → height 1 ✅
- **E:** no children → height 1 ✅
- **C:** no children → height 1 ✅
- **D:** left = 1 (F), right = 0 → diff = 1 ✅ → height 2
- **B:** left = 2 (D), right = 1 (E) → diff = 1 ✅ → height 3
- **A:** left = 3 (B), right = 1 (C) → diff = 2 ❌ **UNBALANCED!**

Step 2 — Which case?

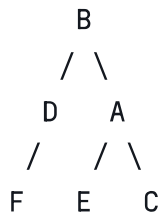
From **A**, the heavy side goes:

- Left to **B**
- Then Left again to **D** (D is heavier than E)

That's **LL**!

Step 3 — Single right rotation on A:

A falls right, B rises up:



Verify:

- **D:** left = 1, right = 0 → diff = 1 ✓
- **A:** left = 1 (E), right = 1 (C) → diff = 0 ✓
- **B:** left = 2 (D), right = 2 (A) → diff = 0 ✓

✓ Perfectly balanced!

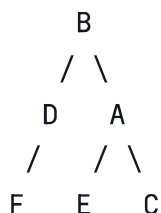
What happen to E

Good catch! E is still D's right child... wait no.

E was **B's right child**. When B rises up to become the new root, E needs a new home. Since:

- B's left = D (stays)
- B's right = A (A fell right)

E has nowhere to go on B anymore! So **E becomes A's left child** because E is smaller than A:



That's exactly what happened — E got **passed down to A** when B rose up. This is actually that **T2** variable in the rotation code:

python

```
def rotate_right(y): # y = A
    x = y.left        # x = B
    T2 = x.right      # T2 = E ← saving E before we lose it!

    x.right = y       # B's right = A
    y.left = T2       # A's left = E ← E finds its new home

    return x          # B is new root
```

So the rotation code explicitly saves E before overwriting B's right pointer, then rehomes it to A's left. That's why **T2** exists — to catch the orphaned child during the rotation!

question, now i have (im in C btw) the AVL made, but the new root of the subtree isnt connected to the rest of the rest of the tree, in contrary its connected on the left/right

Ah classic pointer problem in C! When you rotate, you get the new root back but the **parent still points to the old root**.

So you need to make sure the parent updates its pointer. The cleanest way in C is to **return the new root** all the way up the recursive call:

```
c
Node* insert(Node* node, int key) {
    // normal BST insert
    if (!node) return newNode(key);
    if (key < node->key)
        node->left = insert(node->left, key); // ← reassign!
    else
        node->right = insert(node->right, key); // ← reassign!

    // update height...

    // check balance and rotate
    if (balance > 1 && key < node->left->key)
        return rotate_right(node); // ← return new root up

    if (balance < -1 && key > node->right->key)
        return rotate_left(node); // ← return new root up
```

```
// LR
if (balance > 1 && key > node->left->key) {
    node->left = rotate_left(node->left);
    return rotate_right(node);
}

// RL
if (balance < -1 && key < node->right->key) {
    node->right = rotate_right(node->right);
    return rotate_left(node);
}

return node; // no rotation needed
}
```

The key lines are:

```
c
node->left = insert(node->left, key);
node->right = insert(node->right, key);
```

Because `insert` returns the new root of the subtree, the parent **automatically gets updated** to point to it. So the connection is never lost.

And at the very top level:

```
c
root = insert(root, key); // ← reassign root too!
```

Is that the issue you're running into?

ah.. im looking at it wrong, now i know, i was trying to look at it if i made the avl for a premade tree not a new one that implemented AVL since the beginning

Exactly!! That's the same confusion from earlier with the drawn tree!

AVL trees **build and balance as they go** — so you can't just take a random existing tree and slap AVL rotations on it. The tree was never AVL to begin with so the structure is already "wrong."

If you want AVL, you have two options:

1. **Build from scratch with AVL** — insert nodes one by one using the AVL insert function, it balances automatically as it grows
2. **Convert an existing tree** — you'd have to take all the values out and re-insert them one by one into a fresh AVL tree

Option 2 is basically just:

```
c
// inorder traversal to get sorted values
// then insert each one into a new AVL tree
root = insert(newRoot, value);
```

But nobody really does option 2 in practice — if you need AVL you just build it as AVL from the start.

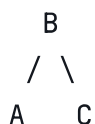
So yeah your confusion made total sense — you were mentally trying to apply AVL logic to a tree that was never built with AVL rules, which is like trying to fix a house that was built without a blueprint by just moving one wall!

```
//need 4 things to check, LL, RR, LR, RL
if (root->left!=NULL && root->left->left!=NULL){ //LL
    return rightRotate(root);
}
if (root->right!=NULL && root->right->right!=NULL){ //RR
    return leftRotate(root);
}
```

Show more

Not quite! The problem is you're checking if those pointers **exist** rather than checking if the tree is actually **unbalanced** there.

For example this tree would trigger your LL check even though it's perfectly balanced:



```

/
X

```

`root->left` exists ✓ and `root->left->left` exists ✓ — but balance factor = 1, no rotation needed!

The correct way is to use the **balance factor**:

```

c
int getBalance(Node* node) {
    if (node == NULL) return 0;
    return height(node->left) - height(node->right);
}

// then in your insert/rebalance:
int balance = getBalance(root);

// LL
if (balance > 1 && getBalance(root->left) >= 0)
    return rightRotate(root);

// RR
if (balance < -1 && getBalance(root->right) <= 0)
    return leftRotate(root);

// LR
if (balance > 1 && getBalance(root->left) < 0) {
    root->left = leftRotate(root->left);
    return rightRotate(root);
}

// RL
if (balance < -1 && getBalance(root->right) > 0) {
    root->right = rightRotate(root->right);
    return leftRotate(root);
}

```

The logic is:

- `balance > 1` → left side too heavy → LL or LR
- `balance < -1` → right side too heavy → RR or RL
- Then check the **child's** balance to determine which subcase

no no, its on a different function, im trying to modularize so a function doesn't have like 10 jobs, but if the root (so the subtree) that is delivered is known to be unbalanced

Oh I see! So the function already **knows** the subtree is unbalanced when it receives it, it just needs to figure out which of the 4 cases it is.

In that case your pointer checking logic is actually closer to correct! But still has an edge case problem. The better check would be the child's balance factor:

```
c
if (getBalance(root->left) >= 0) // LL
    return rightRotate(root);

if (getBalance(root->right) <= 0) // RR
    return leftRotate(root);

if (getBalance(root->left) < 0) { // LR
    root->left = leftRotate(root->left);
    return rightRotate(root);
}

if (getBalance(root->right) > 0) { // RL
    root->right = rightRotate(root->right);
    return leftRotate(root);
}
```

Since the function already knows it's unbalanced, you skip the `balance > 1` or `balance < -1` check entirely. You just need to check the **child's balance** to distinguish:

- LL vs LR (is the left child leaning left or right?)
- RR vs RL (is the right child leaning left or right?)

That's a clean way to modularize it!

